

EUROPEAN UNIVERSITY OF LEFKE
FACULTY OF ENGINEERING
Department of Computer Engineering



COMP343

Signals and Systems

Lab 1-5

Student Name & Surname: Anthony Ogbuah

Student No: 23247090

Department: Computer Engineering

MATLAB Primer

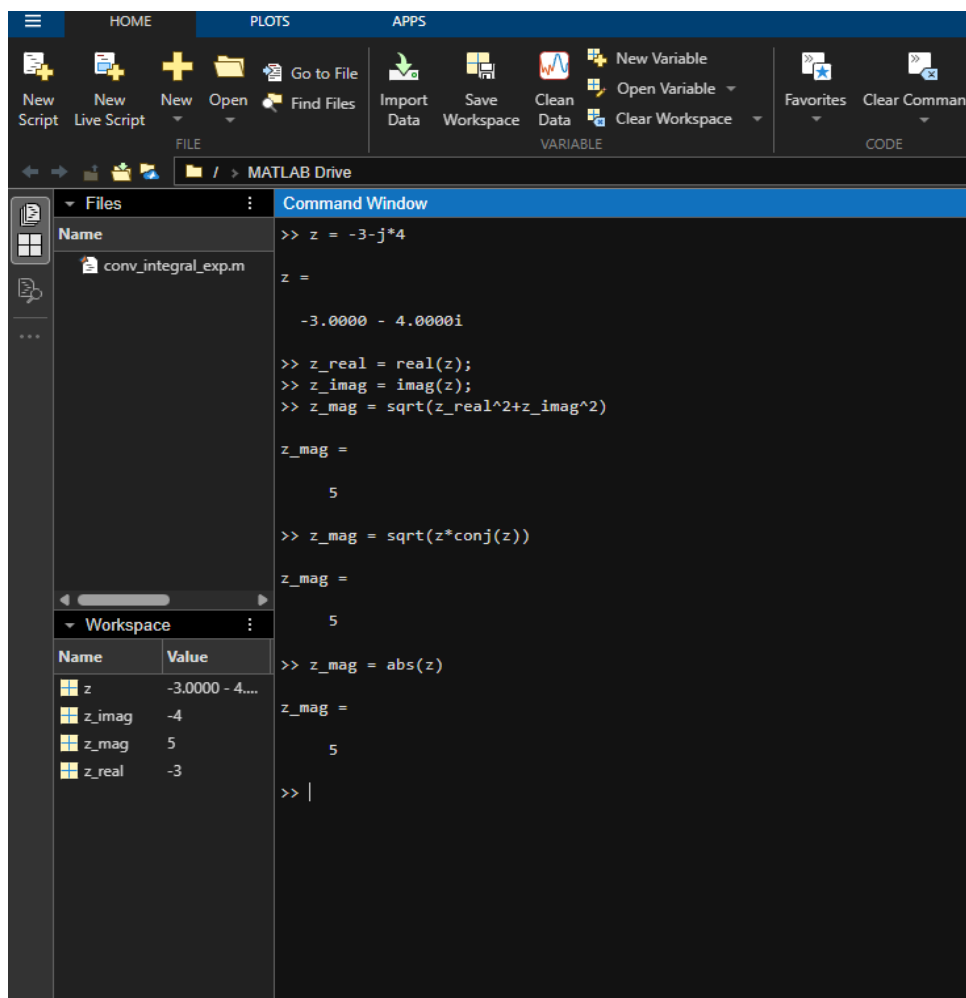
Objectives

To become familiar with the MATLAB environment and its basic functionalities for signal analysis, including the creation and manipulation of numeric arrays, vector representation, and waveform generation, in order to develop foundational skills for modeling, visualization, and analysis of signals.

Results

As shown in the screenshots, MATLAB was successfully used to perform basic operations, including complex number computations, vector and matrix creation, and element-wise operations. Sinusoidal signals were generated and visualized accurately, and indexing allowed access to specific vector and matrix elements. Linear systems were solved using left and right division, and cube roots of complex numbers were calculated correctly. The results demonstrate MATLAB's capability for numerical computation, signal representation, and matrix manipulation.

Basic MATLAB Operations



The screenshot displays the MATLAB environment with the Command Window and Workspace visible. The Command Window shows the execution of a script named `conv_integral_exp.m`. The script defines a complex number `z` and calculates its real, imaginary, and magnitude components.

```
>> z = -3-j*4
z =
-3.0000 - 4.0000i

>> z_real = real(z);
>> z_imag = imag(z);
>> z_mag = sqrt(z_real^2+z_imag^2)

z_mag =
5

>> z_mag = sqrt(z*conj(z))

z_mag =
5

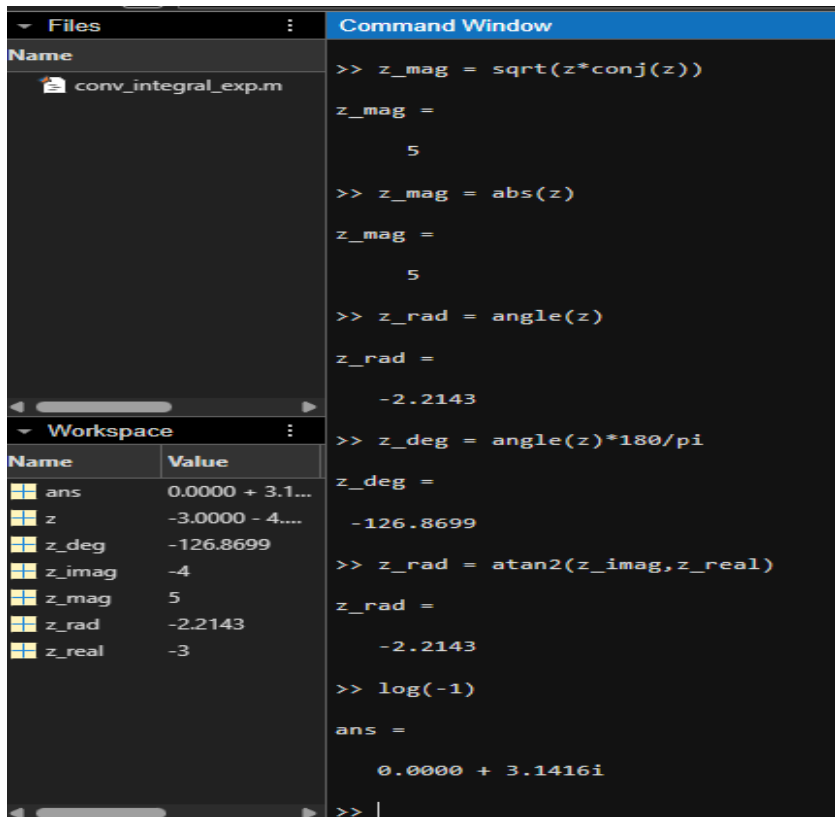
>> z_mag = abs(z)

z_mag =
5

>> |
```

The Workspace window shows the following variables and their values:

Name	Value
z	-3.0000 - 4.0000i
z_imag	-4
z_mag	5
z_real	-3



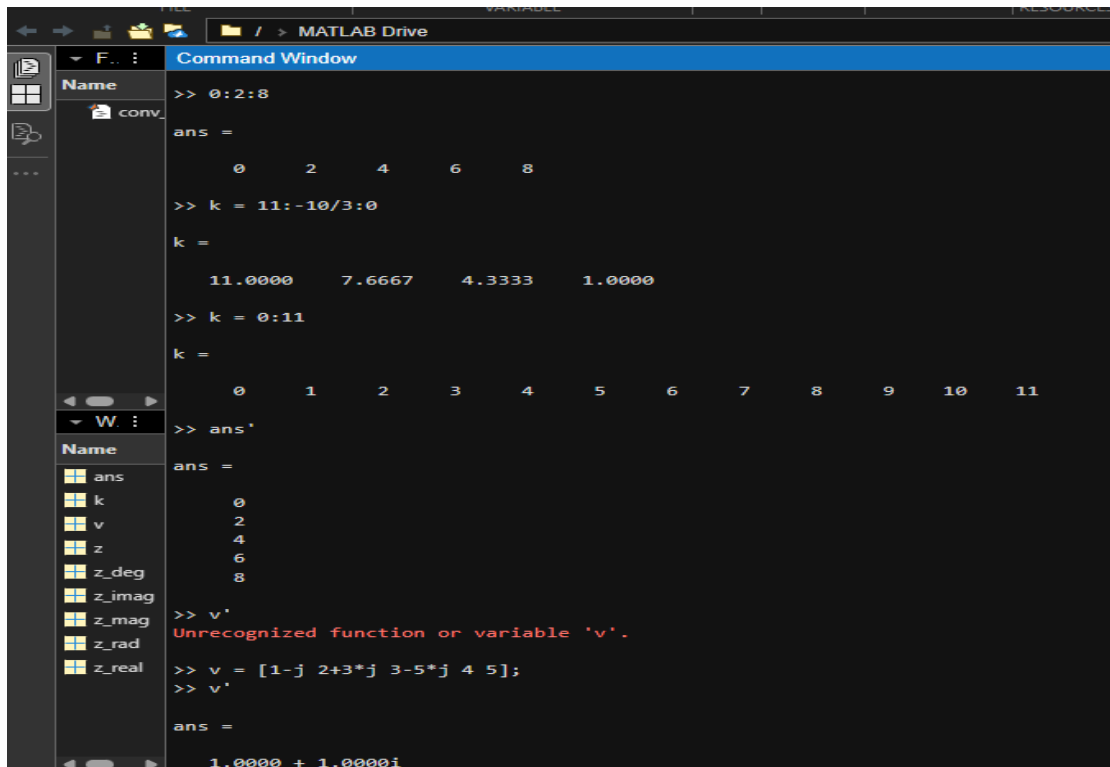
The image shows a MATLAB Command Window and Workspace. The Command Window contains the following code and output:

```
>> z_mag = sqrt(z*conj(z))  
z_mag =  
5  
  
>> z_mag = abs(z)  
z_mag =  
5  
  
>> z_rad = angle(z)  
z_rad =  
-2.2143  
  
>> z_deg = angle(z)*180/pi  
z_deg =  
-126.8699  
  
>> z_rad = atan2(z_imag,z_real)  
z_rad =  
-2.2143  
  
>> log(-1)  
ans =  
0.0000 + 3.1416i  
  
>> |
```

The Workspace shows the following variables and their values:

Name	Value
ans	0.0000 + 3.1416i
z	-3.0000 - 4.0000i
z_deg	-126.8699
z_imag	-4
z_mag	5
z_rad	-2.2143
z_real	-3

Introduction to Vectors in MATLAB

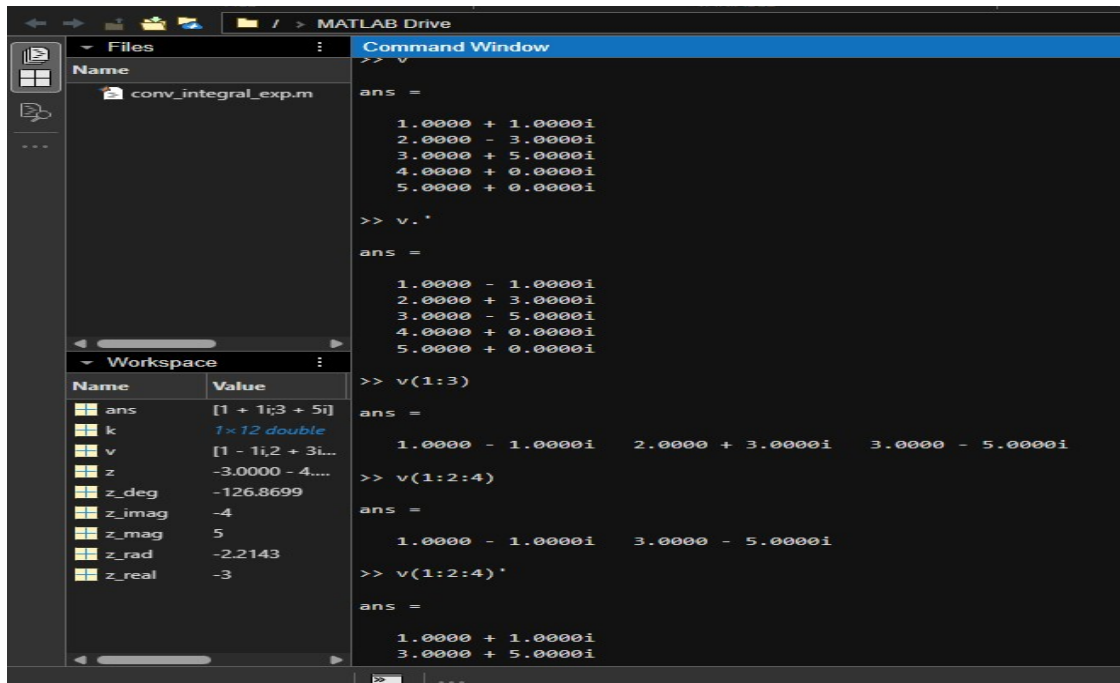


The image shows a MATLAB Command Window and Workspace. The Command Window contains the following code and output:

```
>> 0:2:8  
ans =  
0 2 4 6 8  
  
>> k = 11:-10/3:0  
k =  
11.0000 7.6667 4.3333 1.0000  
  
>> k = 0:11  
k =  
0 1 2 3 4 5 6 7 8 9 10 11  
  
>> ans'  
ans =  
0  
2  
4  
6  
8  
  
>> v'  
Unrecognized function or variable 'v'.  
  
>> v = [1-j 2+3*j 3-5*j 4 5];  
>> v'  
ans =  
1.0000 + 1.0000i
```

The Workspace shows the following variables and their values:

Name	Value
ans	1.0000 + 1.0000i
k	0 1 2 3 4 5 6 7 8 9 10 11
v	1.0000 - 1.0000i 2.0000 + 3.0000i 3.0000 - 5.0000i 4.0000 + 0.0000i 5.0000 + 0.0000i
z	-3.0000 - 4.0000i
z_deg	-126.8699
z_imag	-4
z_mag	5
z_rad	-2.2143
z_real	-3



The image shows a MATLAB interface with a Command Window and a Workspace. The Command Window displays the following commands and results:

```
>> v
ans =
    1.0000 + 1.0000i
    2.0000 - 3.0000i
    3.0000 + 5.0000i
    4.0000 + 0.0000i
    5.0000 + 0.0000i

>> v.'
ans =
    1.0000 - 1.0000i
    2.0000 + 3.0000i
    3.0000 - 5.0000i
    4.0000 + 0.0000i
    5.0000 + 0.0000i

>> v(1:3)
ans =
    1.0000 - 1.0000i    2.0000 + 3.0000i    3.0000 - 5.0000i

>> v(1:2:4)
ans =
    1.0000 - 1.0000i    3.0000 - 5.0000i

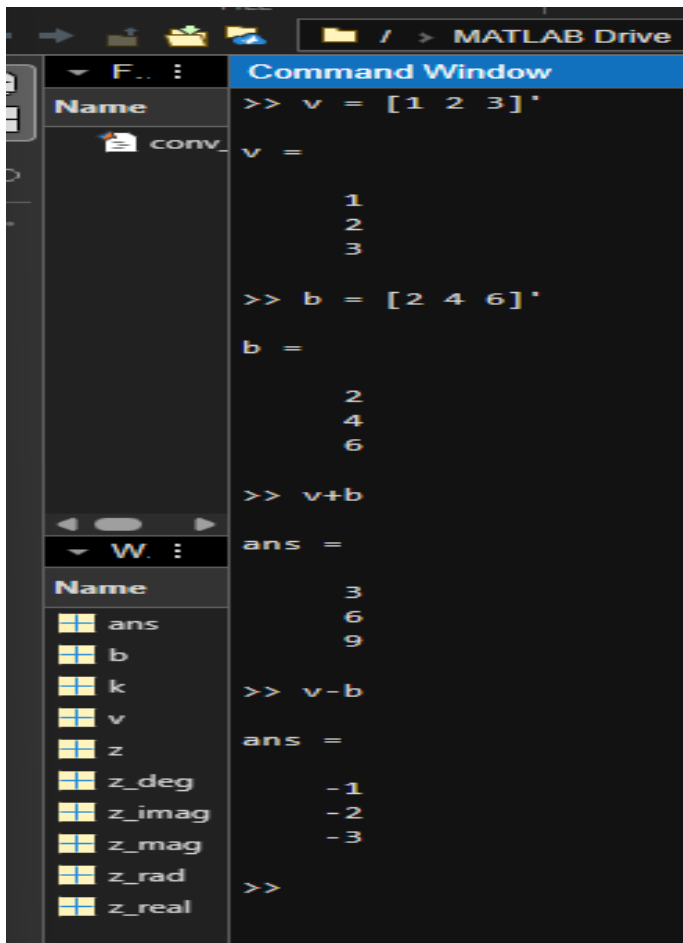
>> v(1:2:4) *
ans =
    1.0000 + 1.0000i
    3.0000 + 5.0000i
```

The Workspace shows the following variables:

Name	Value
ans	[1 + 1i; 3 + 5i]
k	1x12 double
v	[1 - 1i; 2 + 3i; ...]
z	-3.0000 - 4.0000i
z_deg	-126.8699
z_imag	-4
z_mag	5
z_rad	-2.2143
z_real	-3

Vector Functions

Subtraction and Addition



The image shows a MATLAB interface with a Command Window and a Workspace. The Command Window displays the following commands and results:

```
>> v = [1 2 3] *
v =
     1
     2
     3

>> b = [2 4 6] *
b =
     2
     4
     6

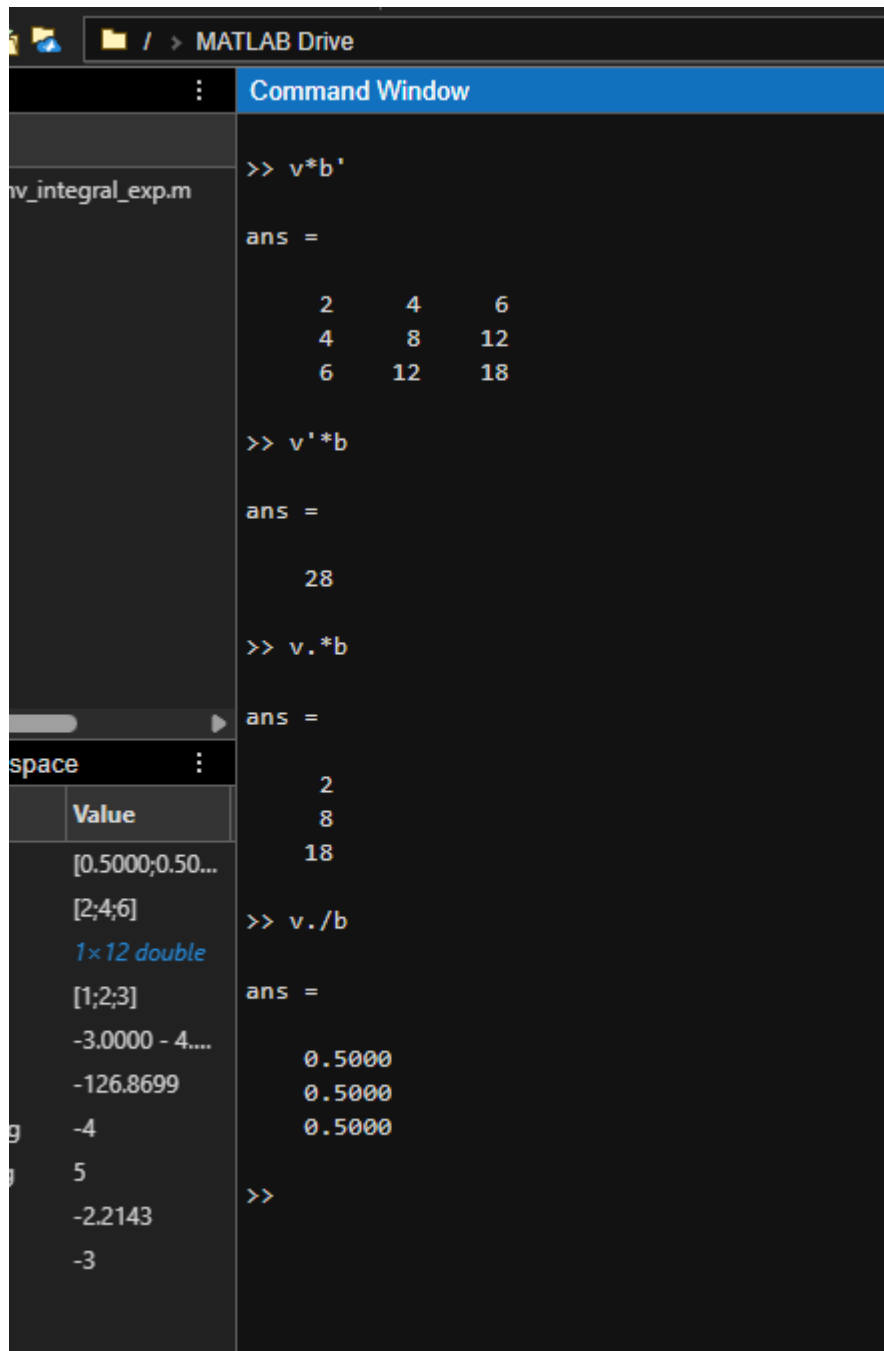
>> v+b
ans =
     3
     6
     9

>> v-b
ans =
    -1
    -2
    -3

>>
```

The Workspace shows the following variables:

Name	Value
ans	[3; 6; 9]
b	[2; 4; 6]
k	1x12 double
v	[1; 2; 3]
z	-3.0000 - 4.0000i
z_deg	-126.8699
z_imag	-4
z_mag	5
z_rad	-2.2143
z_real	-3

Multiplication and Division

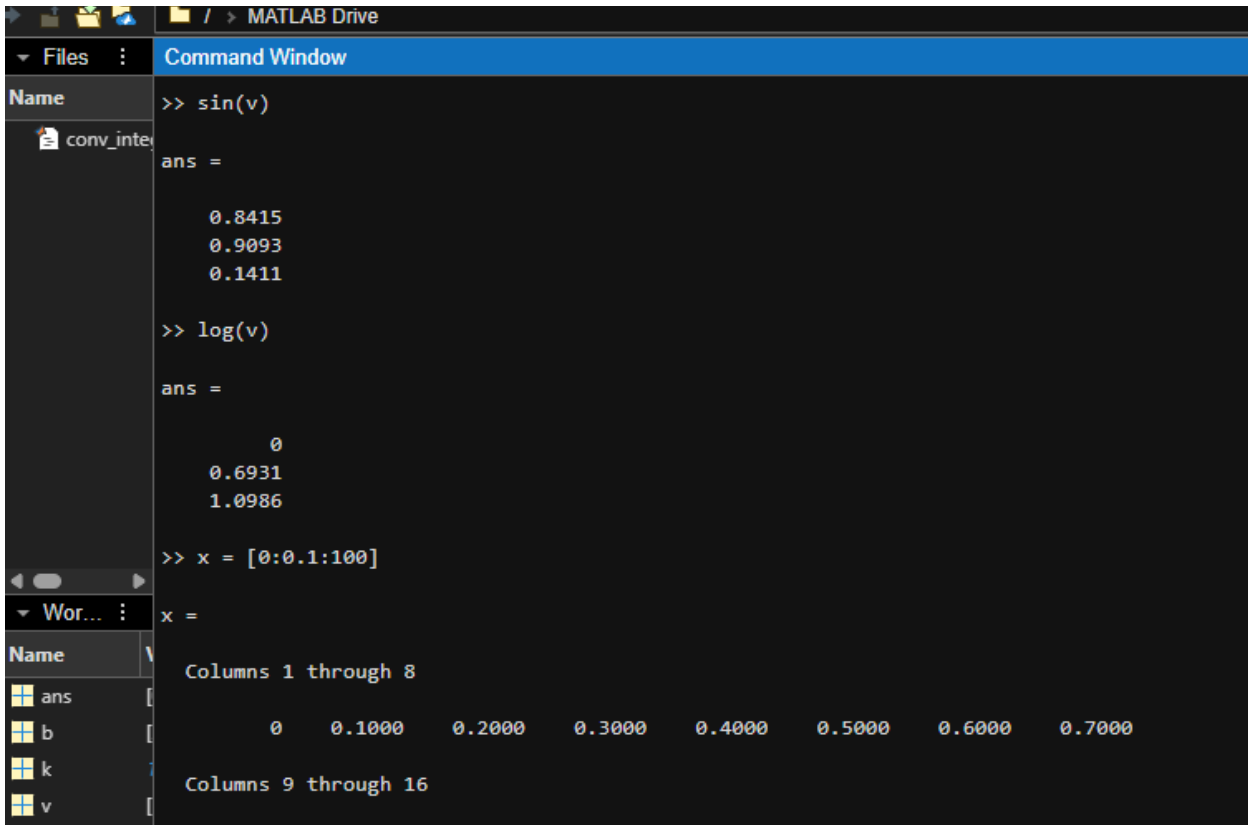
The image shows a MATLAB Command Window with the following content:

```
>> v*b'  
  
ans =  
  
     2     4     6  
     4     8    12  
     6    12    18  
  
>> v'*b  
  
ans =  
  
    28  
  
>> v.*b  
  
ans =  
  
     2  
     8  
    18  
  
>> v./b  
  
ans =  
  
    0.5000  
    0.5000  
    0.5000  
  
>>
```

The left sidebar shows the file explorer with the following files:

- hw_integral_exp.m
- space
- Value
- [0.5000;0.50...
- [2;4;6]
- 1x12 double
- [1;2;3]
- 3.0000 - 4....
- 126.8699
- 4
- 5
- 2.2143
- 3

Non-Linear



```
>> sin(v)

ans =

    0.8415
    0.9093
    0.1411

>> log(v)

ans =

     0
    0.6931
    1.0986

>> x = [0:0.1:100]

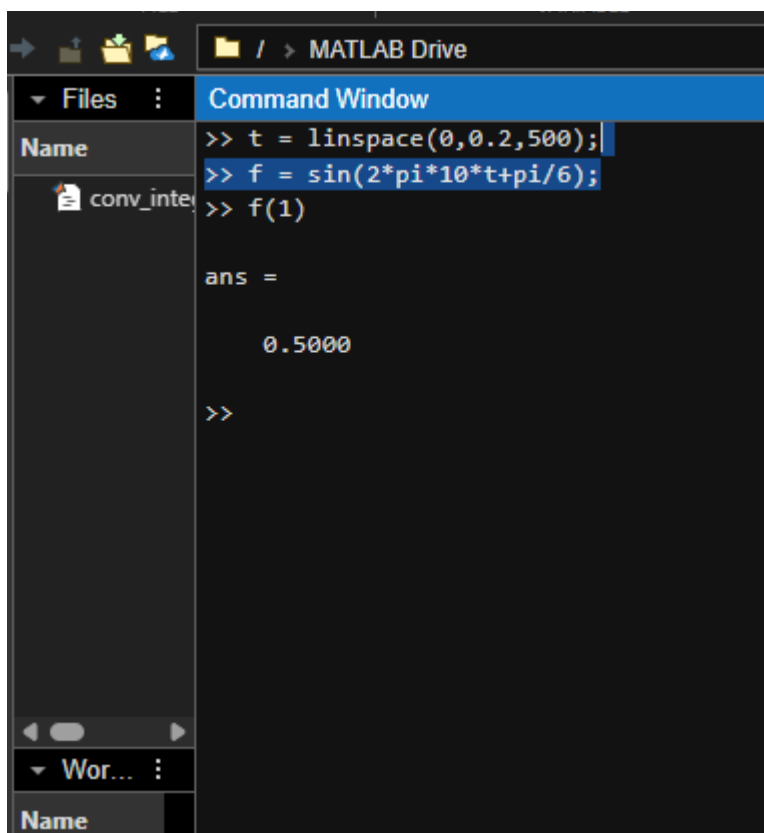
x =

Columns 1 through 8

     0     0.1000     0.2000     0.3000     0.4000     0.5000     0.6000     0.7000

Columns 9 through 16
```

Sinusoids



```
>> t = linspace(0,0.2,500);
>> f = sin(2*pi*10*t+pi/6);
>> f(1)

ans =

    0.5000

>>
```

```

/ > MATLAB Drive
Command Window
Name
conv_inte
>> A = [1 2 3; 3 4 5; 6 7 8]
A =
     1     2     3
     3     4     5
     6     7     8

>> B = [[1 2 3]' [2 4 7]' [3 5 8]']
B =
     1     2     3
     2     4     5
     3     7     8

>> v = [0:2:8]
v =
     0     2     4     6     8

>> inv(A)
Warning: Matrix is close to singular or badly scaled. Results may be inaccurate. RCOND = 3.469447e-18.
ans =
1.0e+15 *
    -2.7022    4.5036   -1.8014
     5.4043   -9.0072    3.6029
    -2.7022    4.5036   -1.8014

>> eig(A)

```

```

/ > MATLAB Drive
Command Window
>> eig(A)
ans =
    14.0664
    -1.0664
     -0.0000

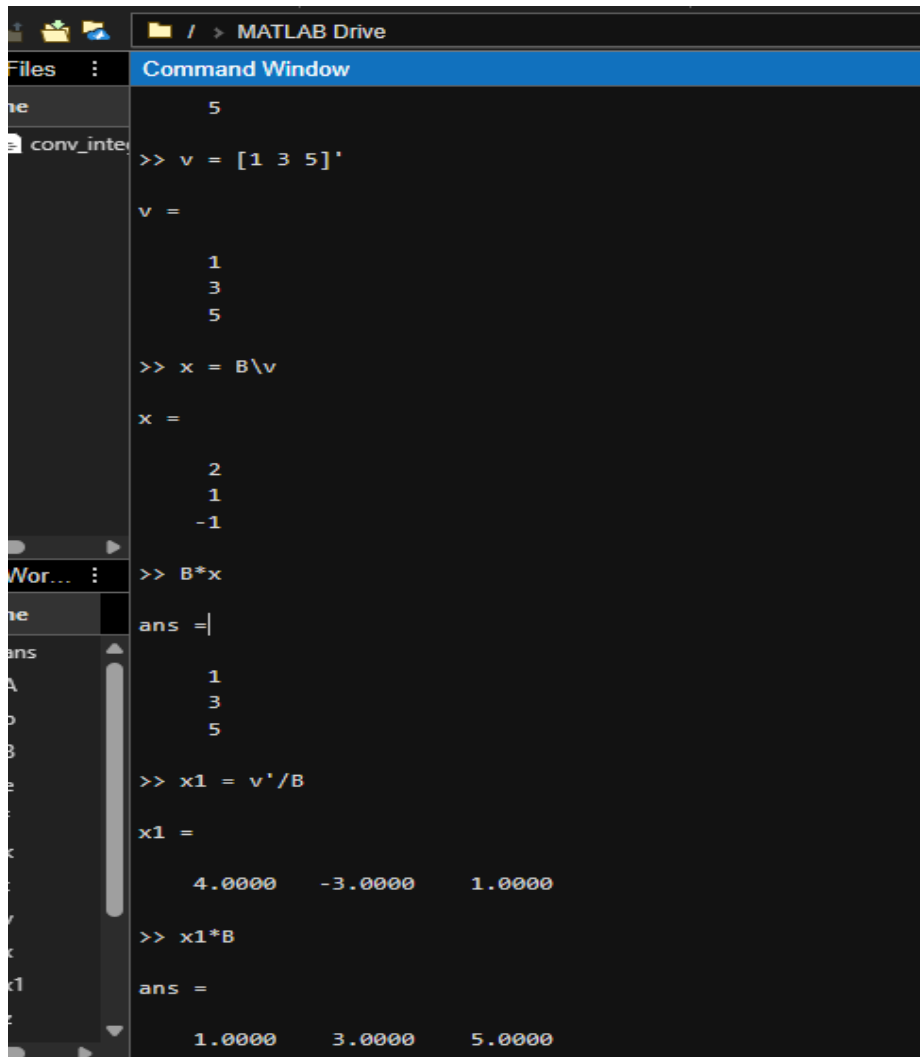
>> [v,e] = eig(A)
v =
    0.2656    0.7444    0.4082
    0.4912    0.1907   -0.8165
    0.8295   -0.6399    0.4082

e =
    14.0664     0     0
         0   -1.0664     0
         0     0   -0.0000

>> diag(e)
ans =
    14.0664
    -1.0664
    -0.0000

>> v = [1 3 5]'
v =
     1
     3
     5

```



The screenshot shows the MATLAB Command Window with the following commands and outputs:

```
>> v = [1 3 5]'
```

v =

```
1
3
5
```

```
>> x = B\v
```

x =

```
2
1
-1
```

```
>> B*x
```

ans =

```
1
3
5
```

```
>> x1 = v'/B
```

x1 =

```
4.0000 -3.0000 1.0000
```

```
>> x1*B
```

ans =

```
1.0000 3.0000 5.0000
```

Conclusion:

The objective was met. It provided a solid introduction to the MATLAB environment. I learned to use the Workspace, navigate the Command Window, and carry out basic computations with scalars and complex numbers. I also learned to construct and manipulate vectors and matrices, as well as differentiate element-wise operations from standard array operations.

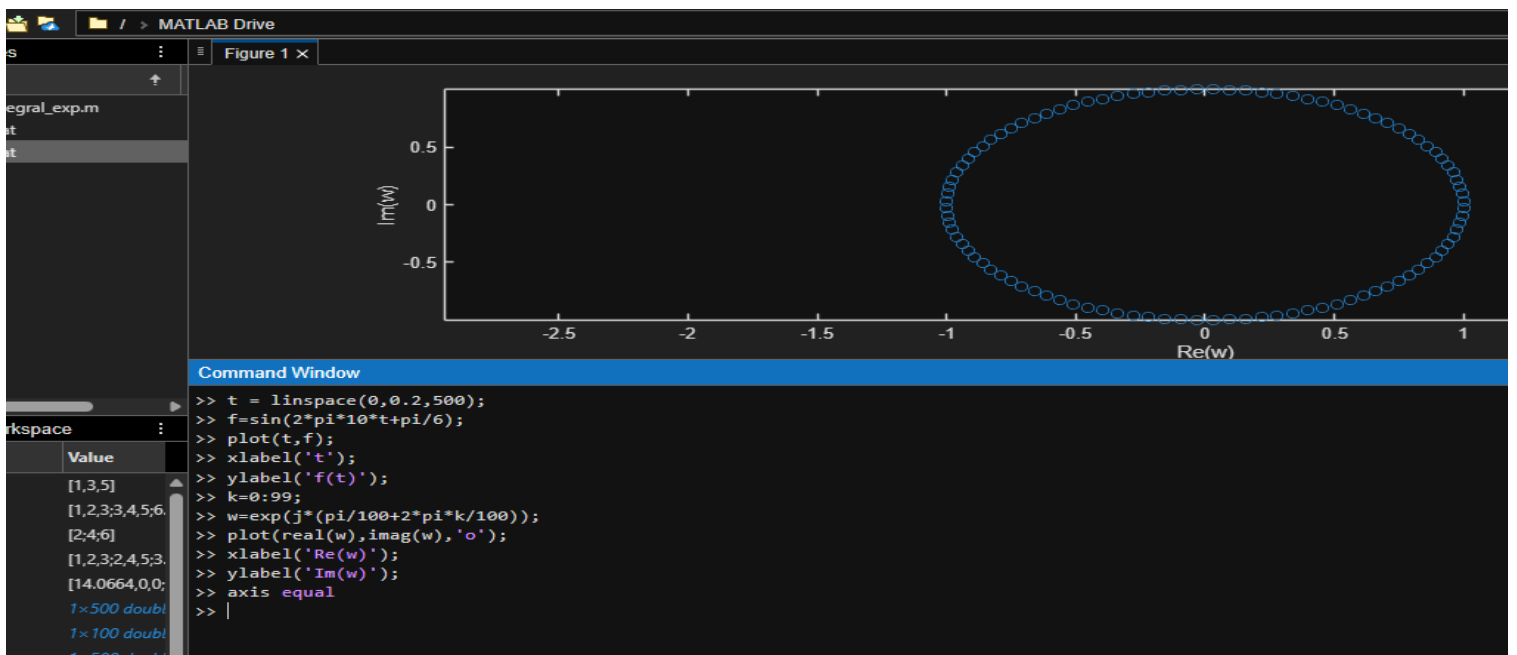
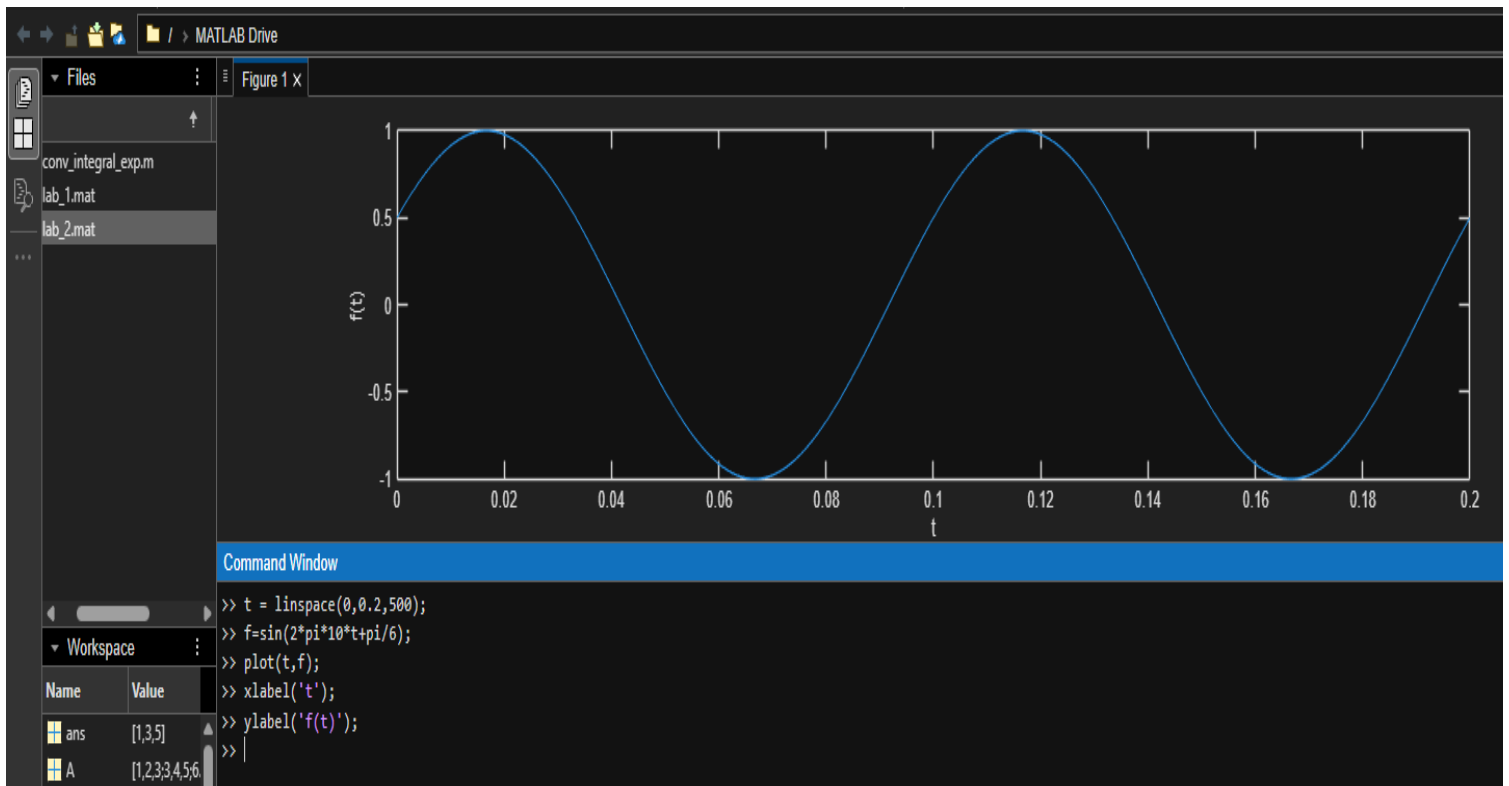
Name & Std. No: Anthony Ogbuah (23247090)

Course: COMP343 Signals and Systems

Assistant: Ferhat Cemoğlu

EXPERIMENT 2**Signal Generation, Manipulation and Visualization I****Objectives**

To learn how to generate, manipulate, and visualize signals in MATLAB, including plotting continuous and discrete signals, performing element-by-element operations, and creating/manipulating matrices for signal analysis.



Matrix Operations

```
Command Window
>> r=[1 0 0]

r =

     1     0     0

>> A=[2 3; 4 5; 0 6]

A =

     2     3
     4     5
     0     6

>> c=r'

c =

     1
     0
     0

>> B=[c A]

B =

     1     2     3
     0     4     5
     0     0     6

>> B(1,2)

ans =

     2

>> B(1:2,2:3)
```

```
MATLAB Drive
Command Window

     0     4     5
     0     0     6

>> B(1,2)

ans =

     2

>> B(1:2,2:3)

ans =

     2     3
     4     5

>> B(2,:)

ans =

     0     4     5

>>
```

Conclusion:

The experiment was completed successfully. We generated and plotted signals, applied element-wise operations to adjust their characteristics, and practiced creating and manipulating matrices in MATLAB. These skills form the basic groundwork for analyzing and interpreting more advanced signal-processing concepts.

Name & Std. No: Anthony Ogbuah (23247090)

Course: COMP343 Signals and Systems

Assistant: Ferhat Cemoğlu

EXPERIMENT 3

Signal Generation, Manipulation and Visualization II

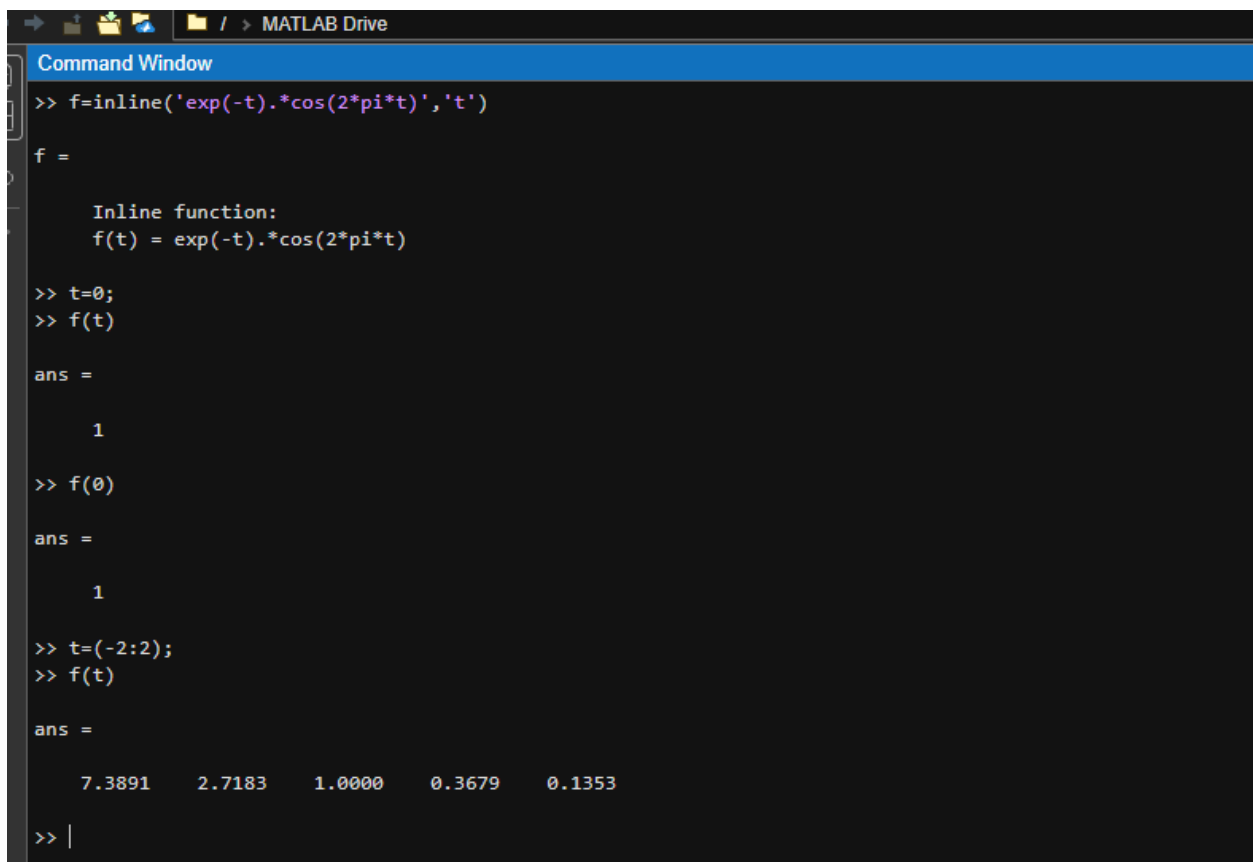
Objective

To learn advanced methods of signal generation, manipulation, and visualization in MATLAB by using inline functions, relational operators, logical operations, and improved sampling techniques for accurately representing signals such as damped sinusoids, unit step functions, and unit pulses.

Results

As shown in the screenshots, MATLAB inline functions enabled accurate generation and plotting of signals like the damped sinusoid, unit step, and unit pulse. Increasing the time resolution improved visualization of oscillations and discontinuities, and relational and logical operators allowed flexible signal definition and manipulation.

Inline Functions



```

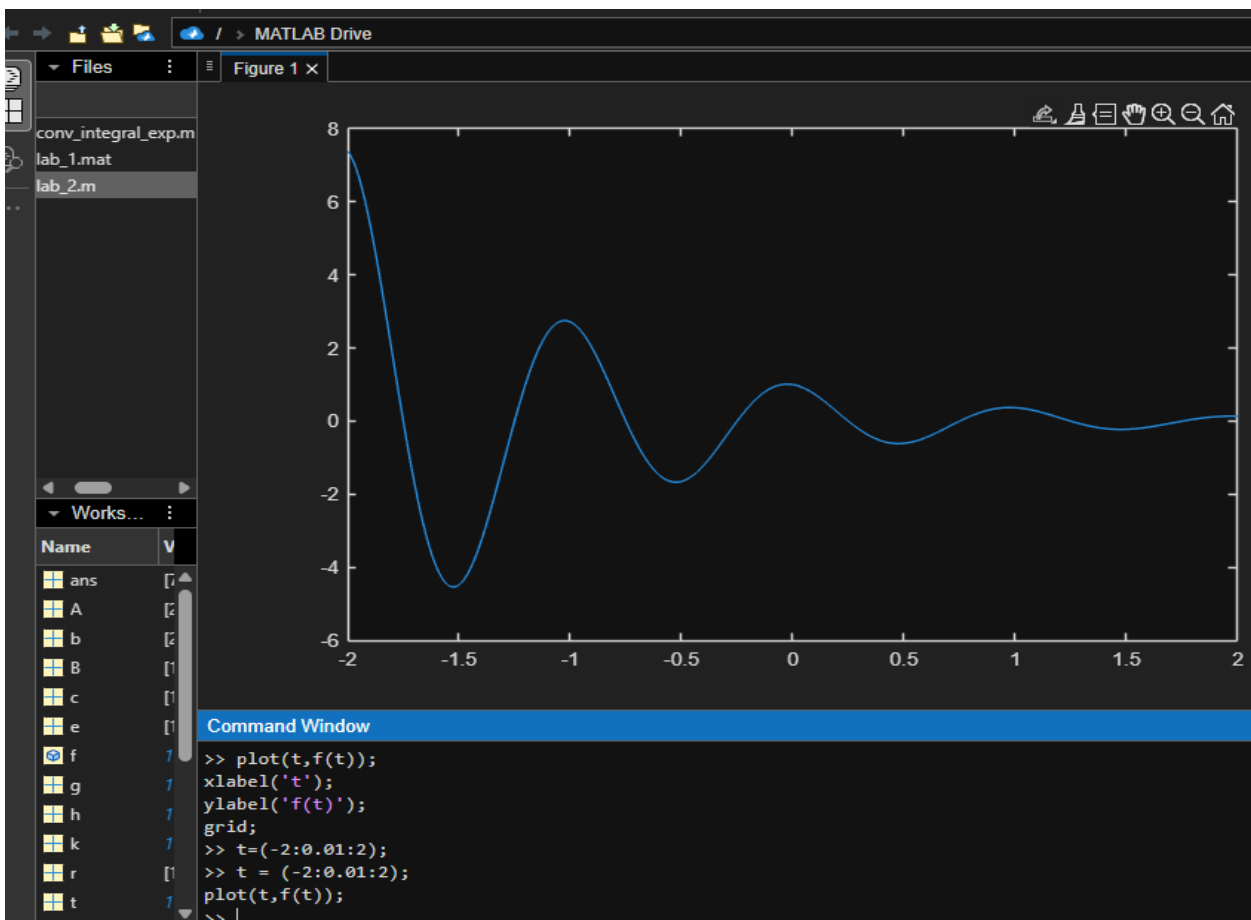
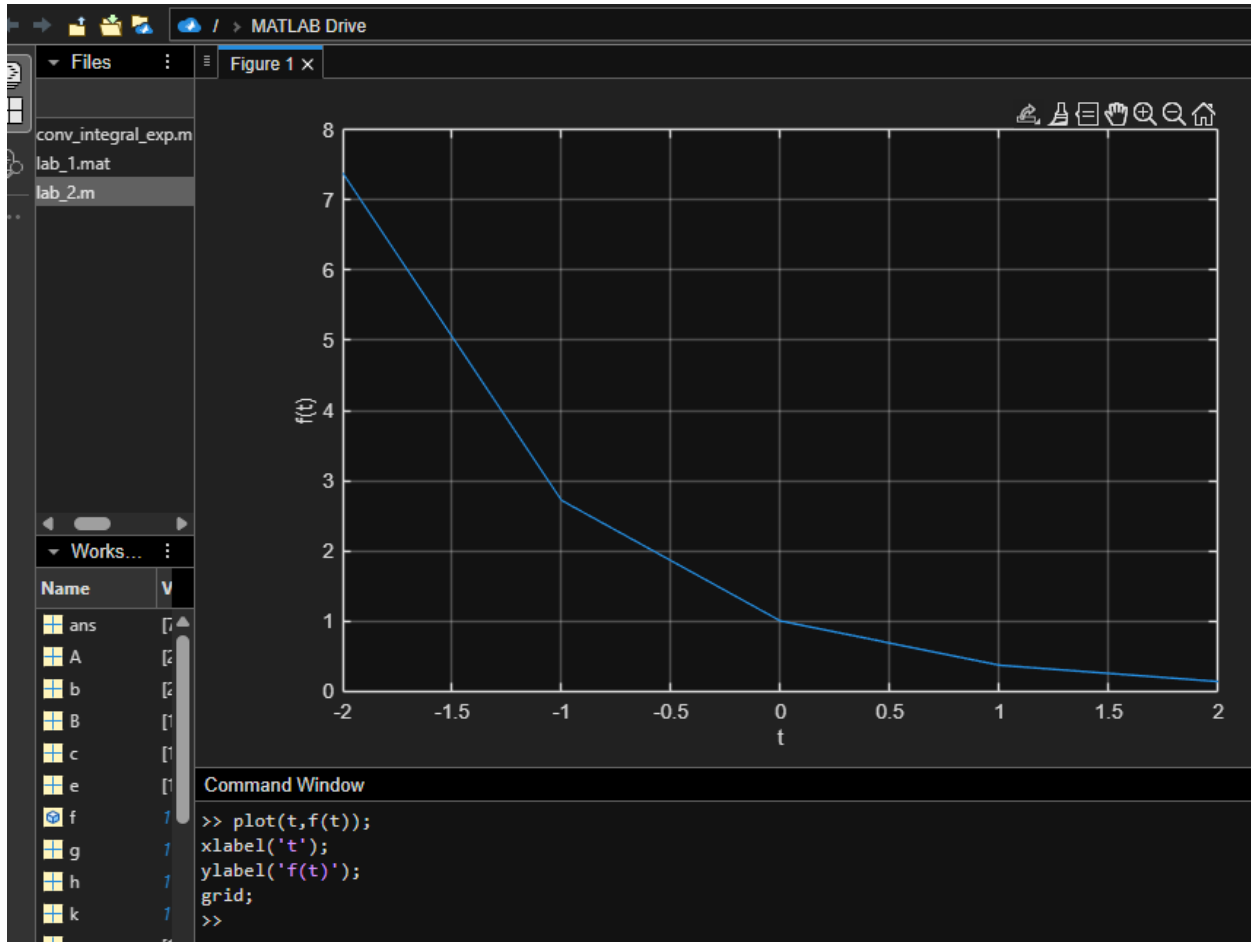
>> f=inline('exp(-t).*cos(2*pi*t)','t')
f =
    Inline function:
    f(t) = exp(-t).*cos(2*pi*t)

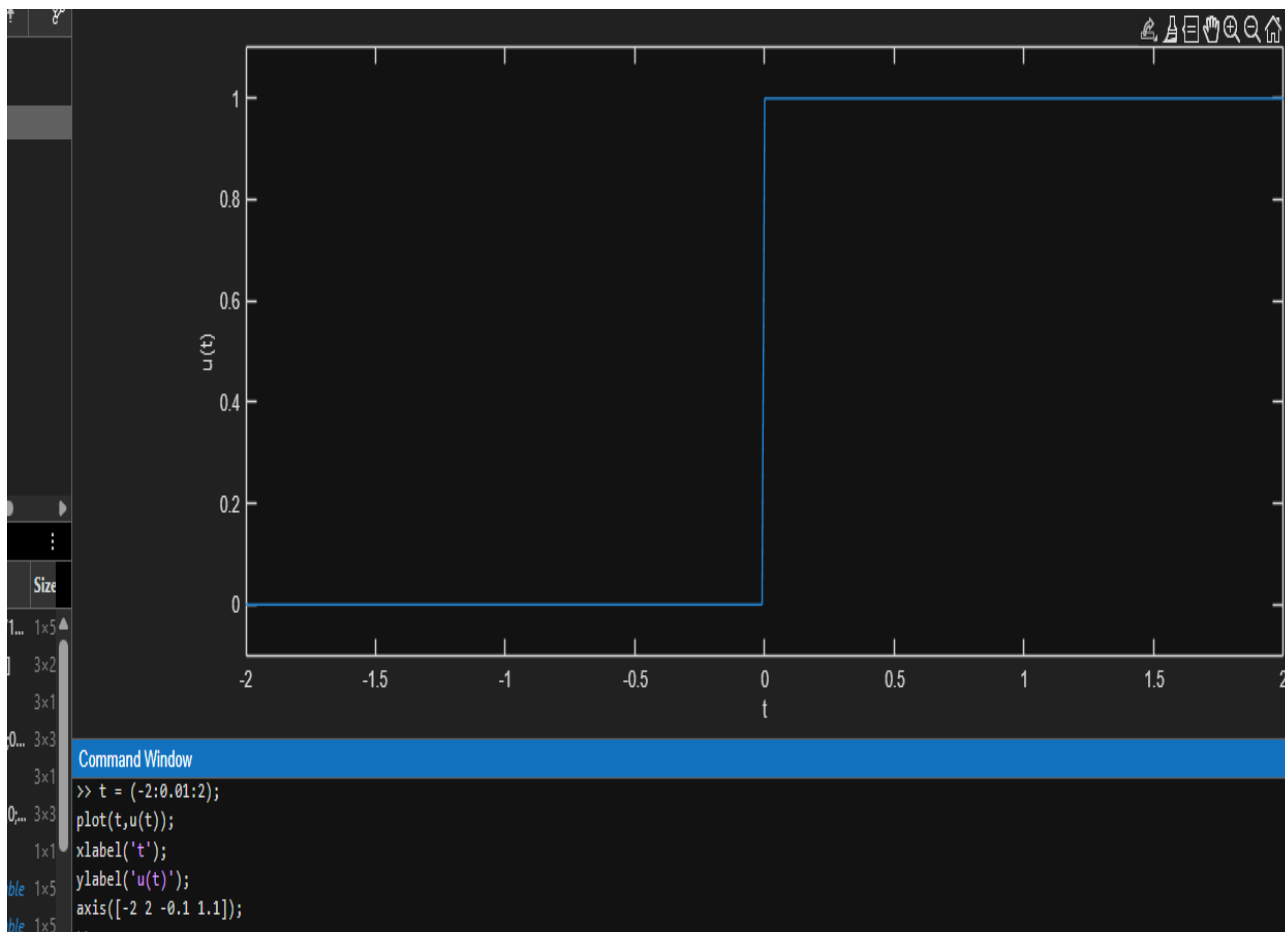
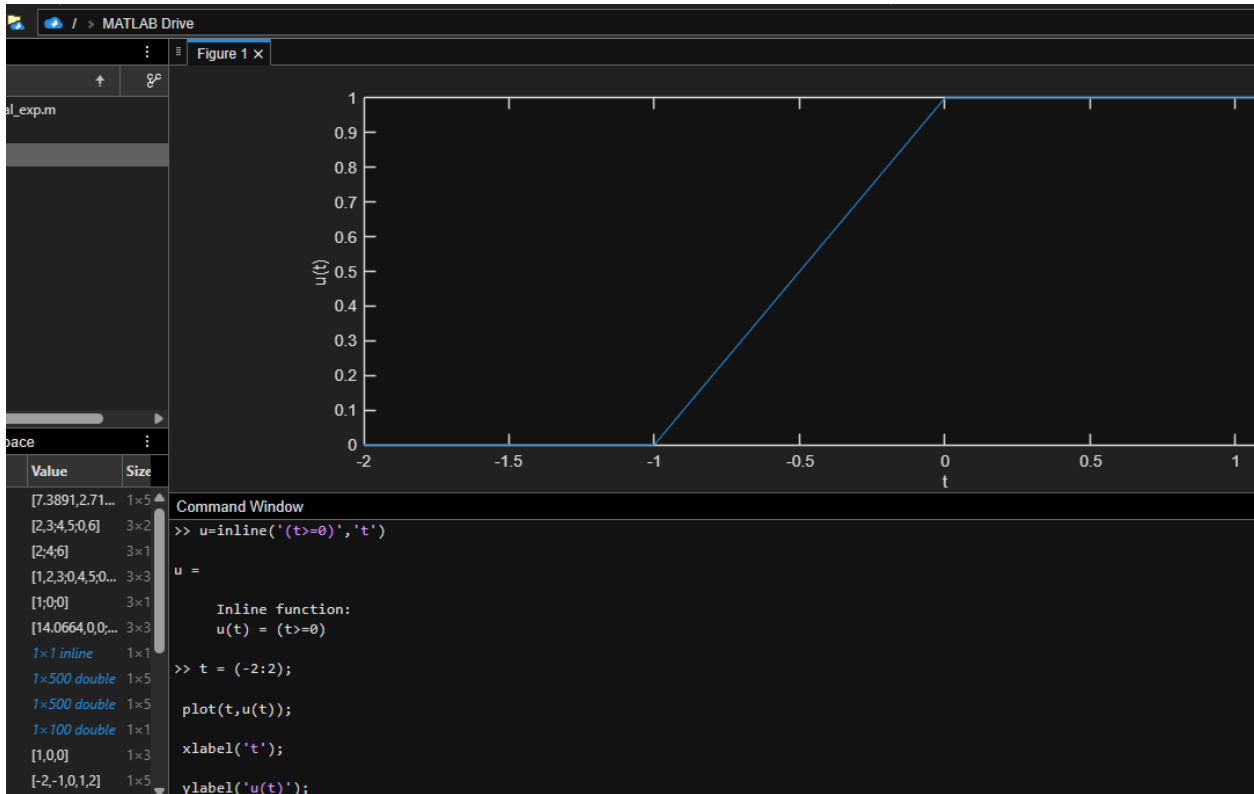
>> t=0;
>> f(t)
ans =
     1

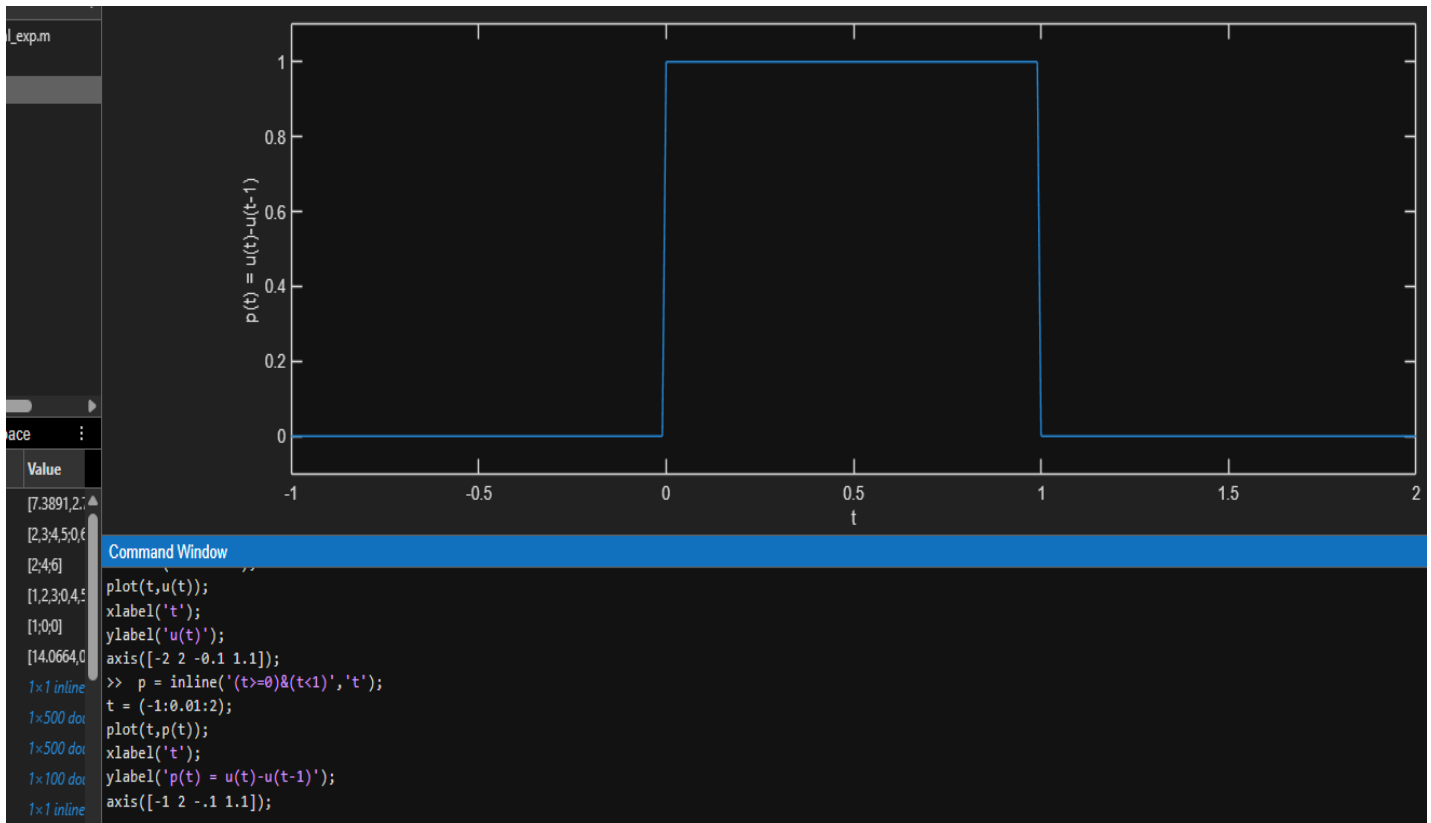
>> f(0)
ans =
     1

>> t=(-2:2);
>> f(t)
ans =
    7.3891    2.7183    1.0000    0.3679    0.1353

>> |
```







Conclusion:

The experiment was completed successfully. We used advanced MATLAB tools to define and evaluate functions, generate accurate signal plots with proper sampling, and model discontinuous signals through relational and logical operations. These methods improve the precision of signal representation and provide the groundwork for more advanced signal analysis and system modeling.

Name & Std. No: Anthony Ogbuah (23247090)

Course: COMP343 Signals and Systems

Assistant: Ferhat Cemoğlu

EXPERIMENT 4

Convolution and Linear Time-Invariant (LTI) System

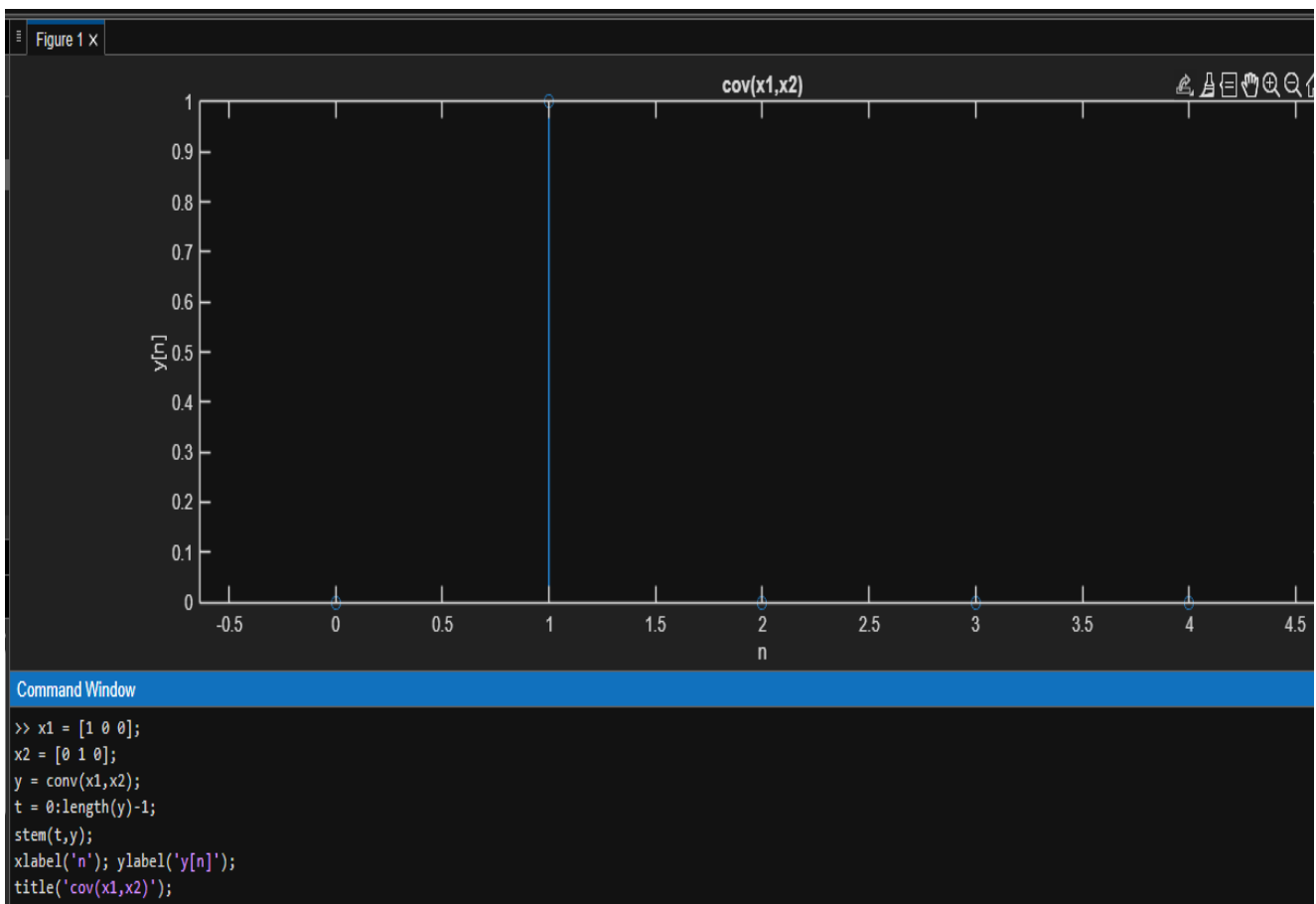
Objective

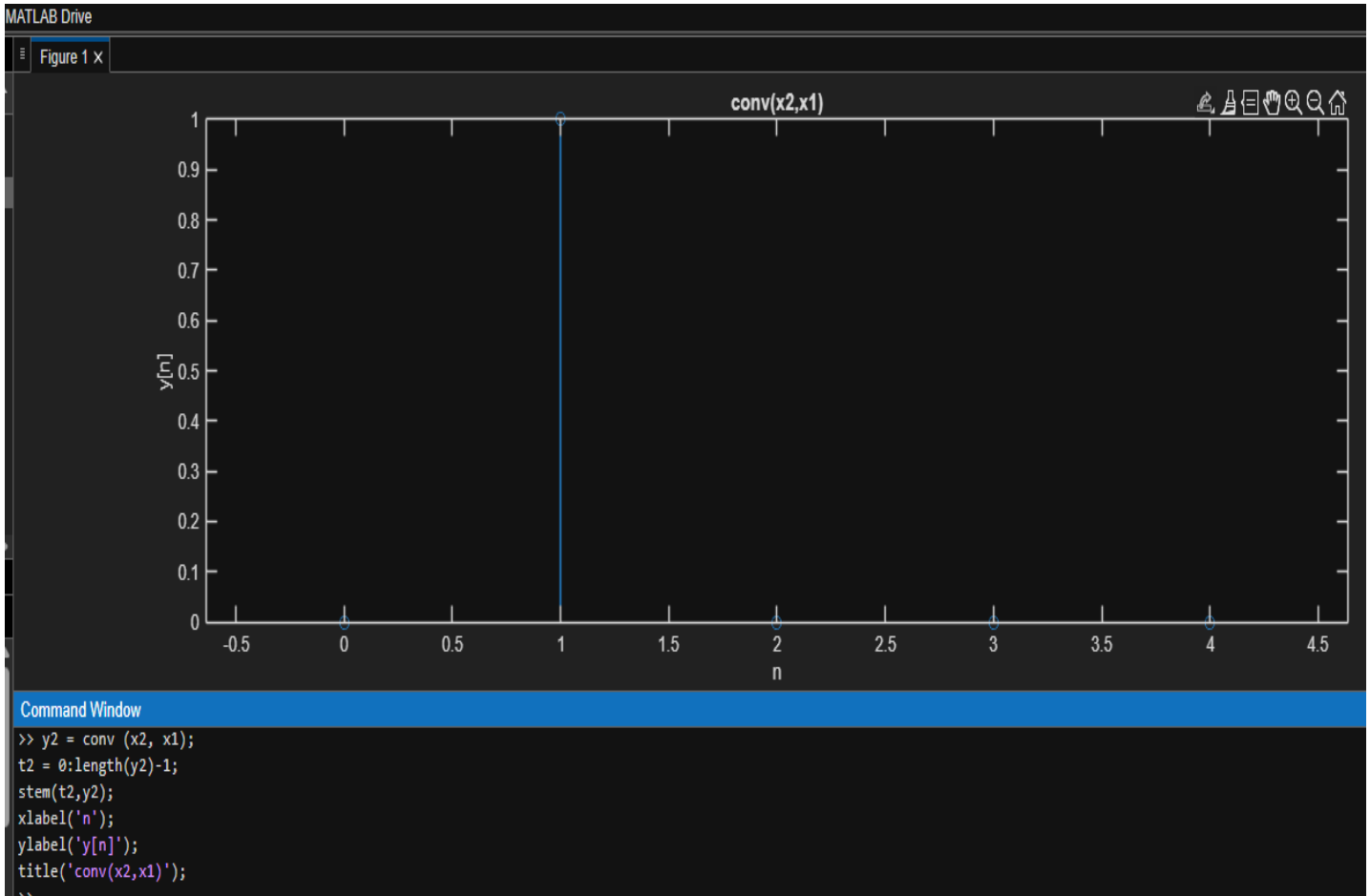
To comprehend and implement the convolution operation in discrete-time signal processing, and to utilize MATLAB for computing and visualizing convolution results to analyze the responses of Linear Time-Invariant (LTI) systems.

Results

The screenshots demonstrate that convolving two impulses produced a shifted impulse, while convolving an impulse with a square wave resulted in a shifted square wave. Convolution of two identical square waves generated a triangular pulse, reflecting the overlapping of samples. All MATLAB outputs aligned with theoretical predictions.

Convolution of two impulses:



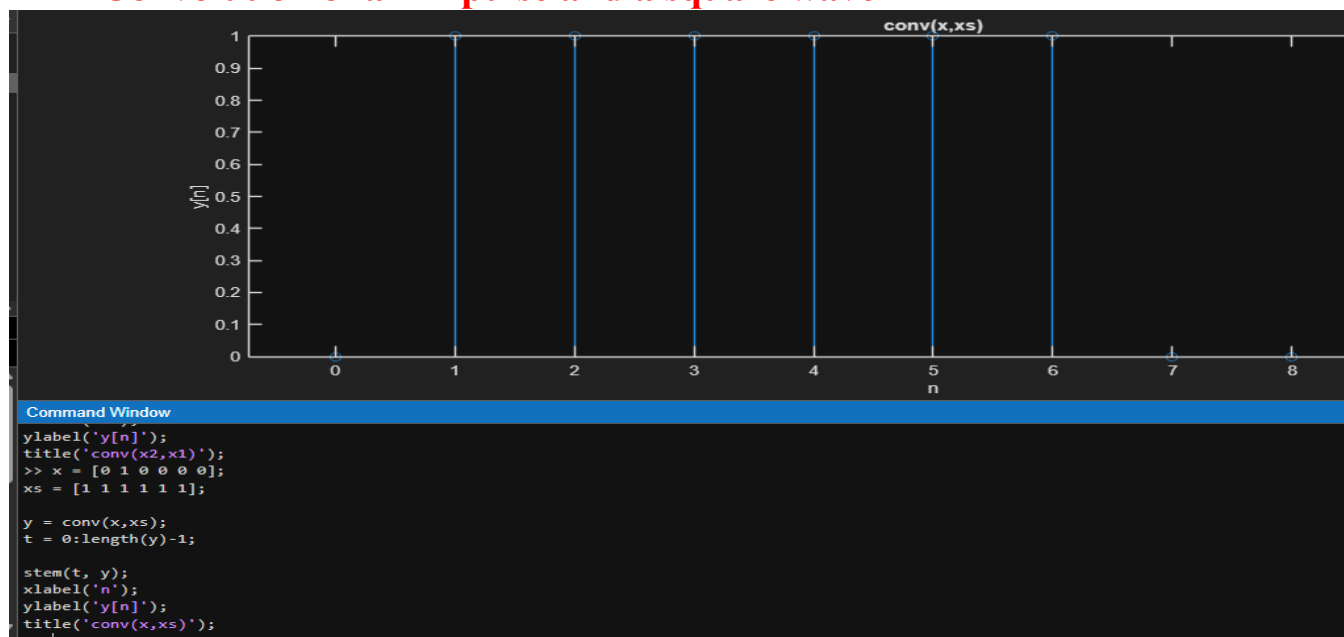


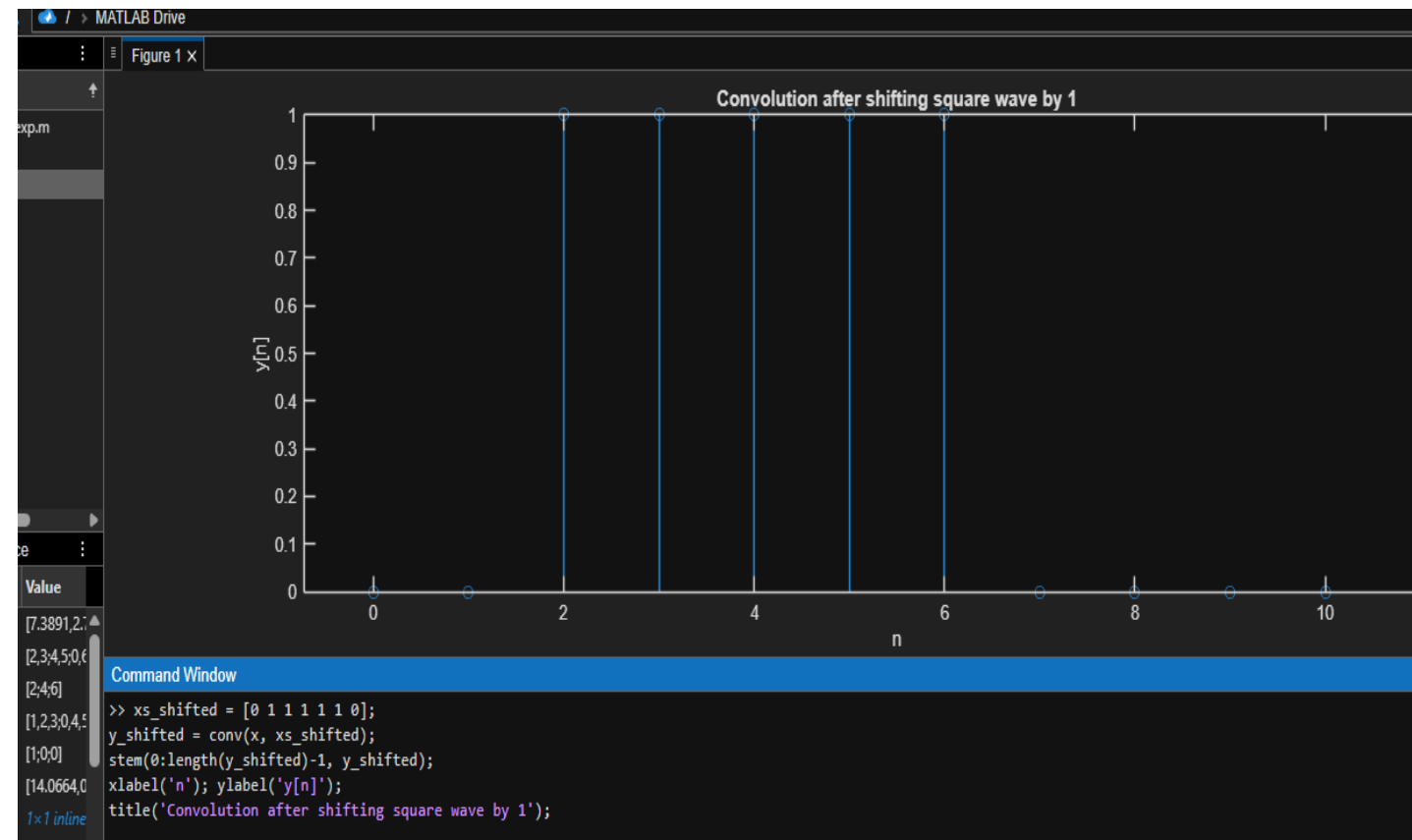
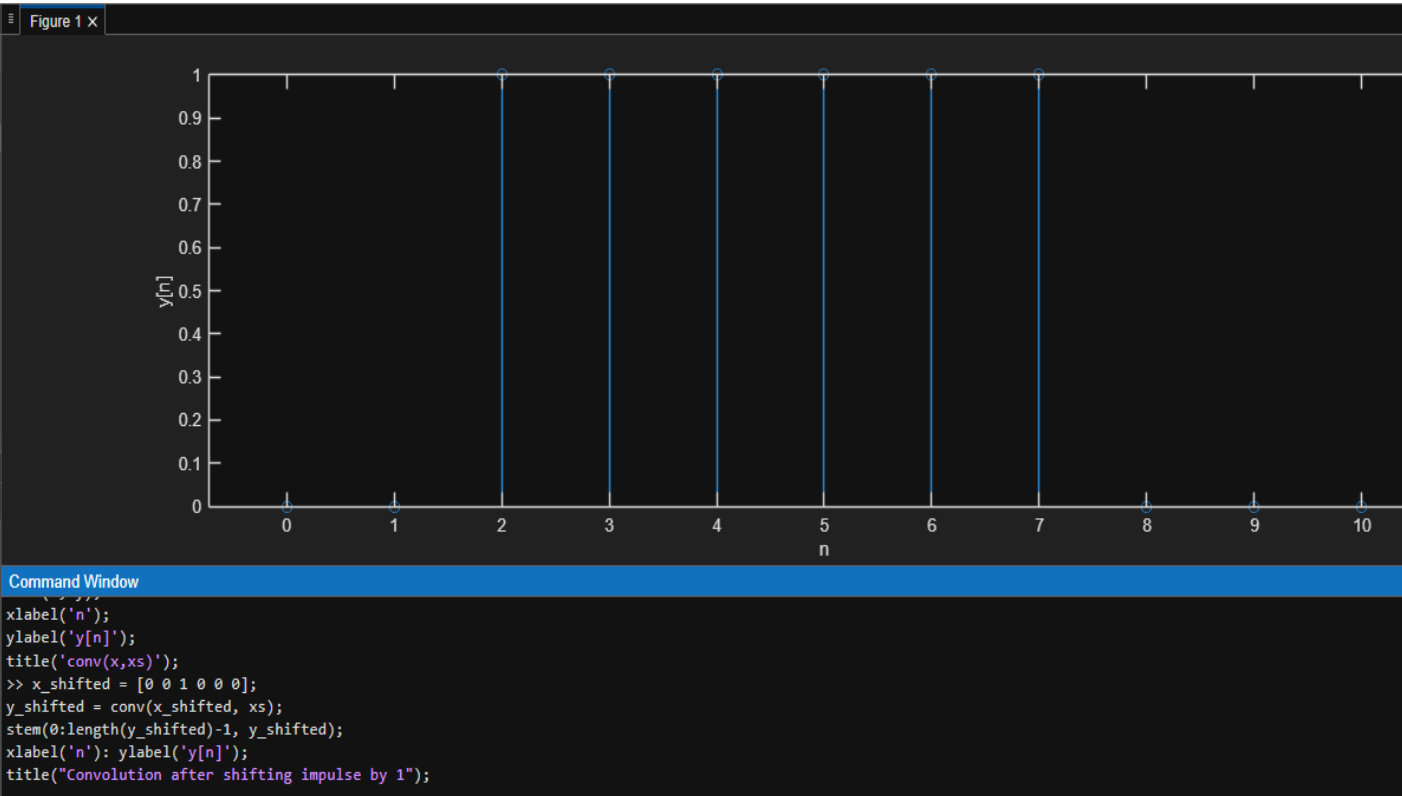
Observations

Convolution of two impulses produces another impulse located at the time index equal to the sum of their positions.

- Any shift in one of the impulses results in the same shift appearing in the output.
- Reversing the order of convolution does not change the output (convolution is commutative).

Convolution of an impulse and a square wave



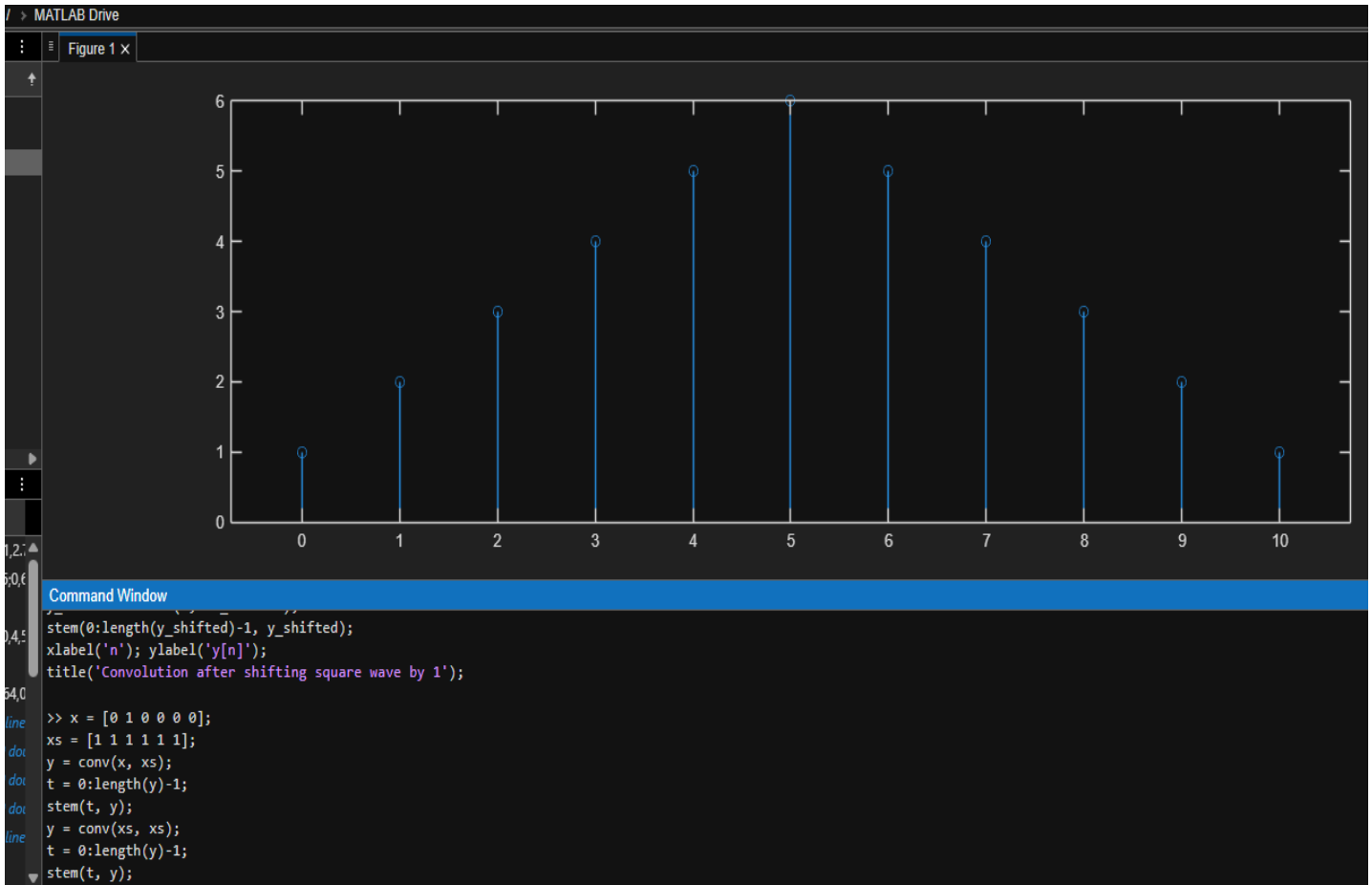


Observation

Convolving an impulse with a square wave reproduces the square wave shifted to the position of the impulse.

Changing the time shift of either the impulse or the square wave simply shifts the output without altering its shape

Convolution of an impulse and a square wave



Observation

Convolution of two identical length-6 square waves produces a triangular pulse of length 11, with values rising linearly to a peak of 6 and then decreasing linearly back to 1. This demonstrates that convolution effectively counts the number of overlapping samples between the sequences.

Conclusion

This experiment utilized MATLAB to perform convolution on impulses, shifted signals, and square waves, validating essential LTI system properties, including time-shifting and commutativity. The results showed that an LTI system's output is defined by the convolution of the input with its impulse response, highlighting convolution as a fundamental method for analyzing and predicting system behavior.

Name & Std. No: Anthony Ogbuah (23247090)

Course: COMP343 Signals and Systems

Assistant: Ferhat Cemoğlu

EXPERIMENT 5

Discrete-time LTI Systems

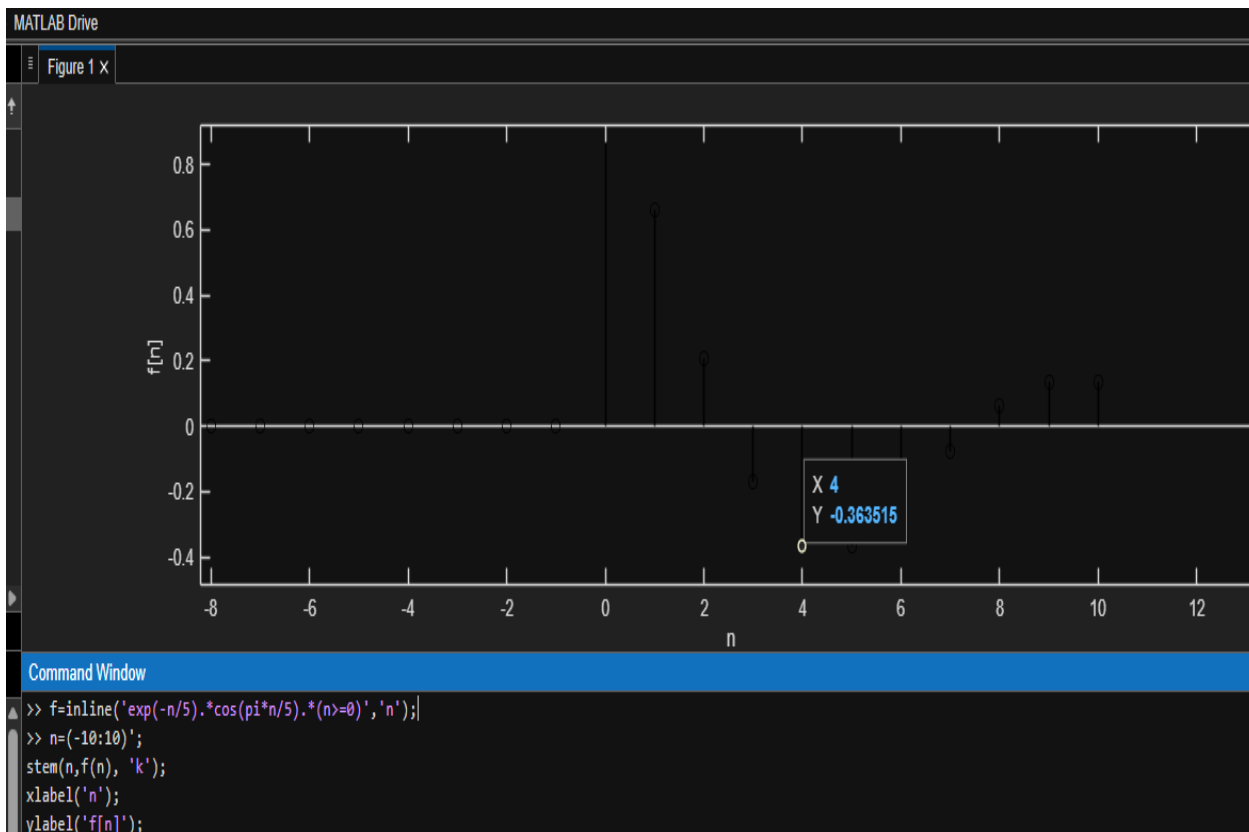
Objective

The experiment aims to generate and analyze discrete-time signals and to study the behavior of discrete-time LTI systems using MATLAB. This involves visualizing sequences with stem plots and employing the `filter` function to compute zero-state, zero-input, and total system responses from a given difference equation.

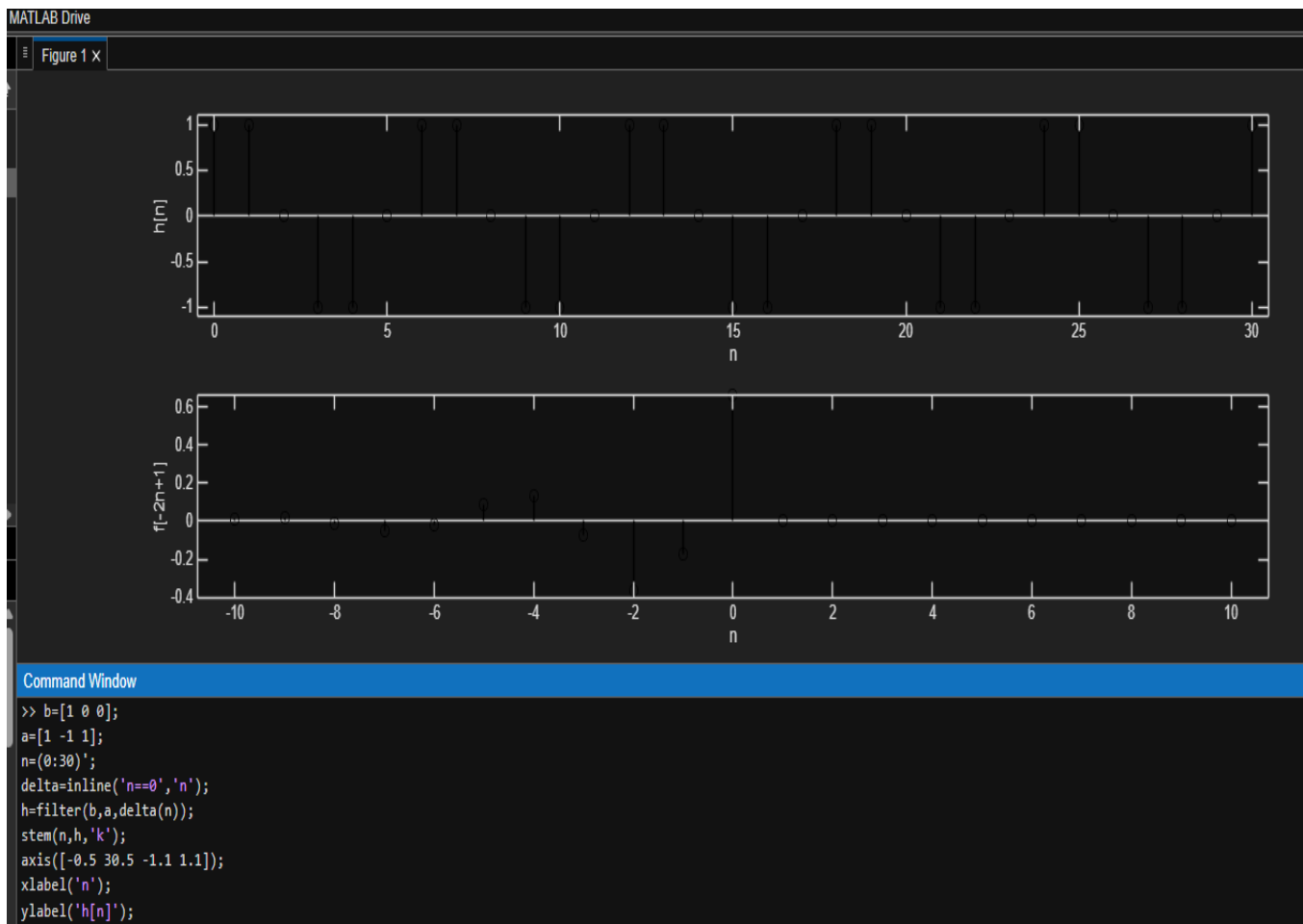
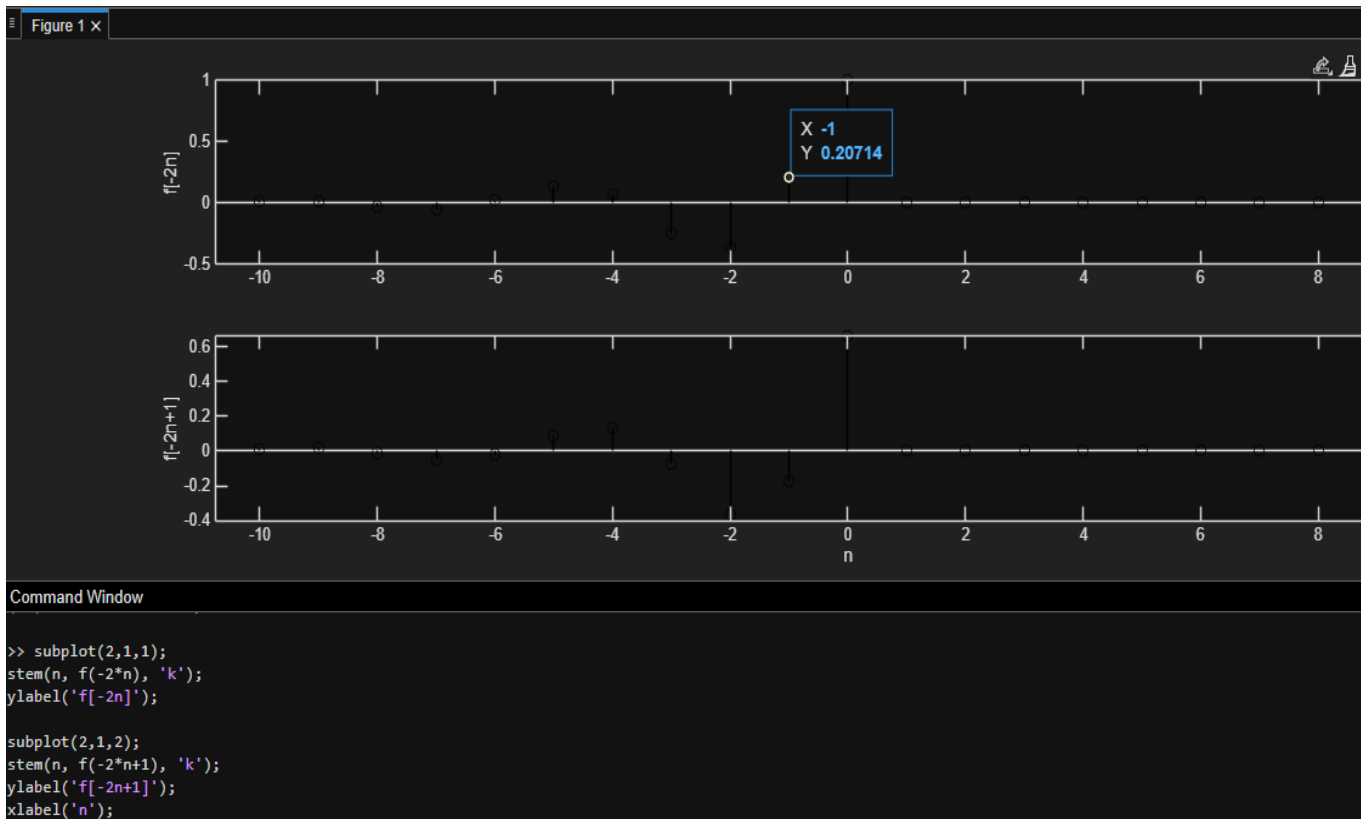
Results

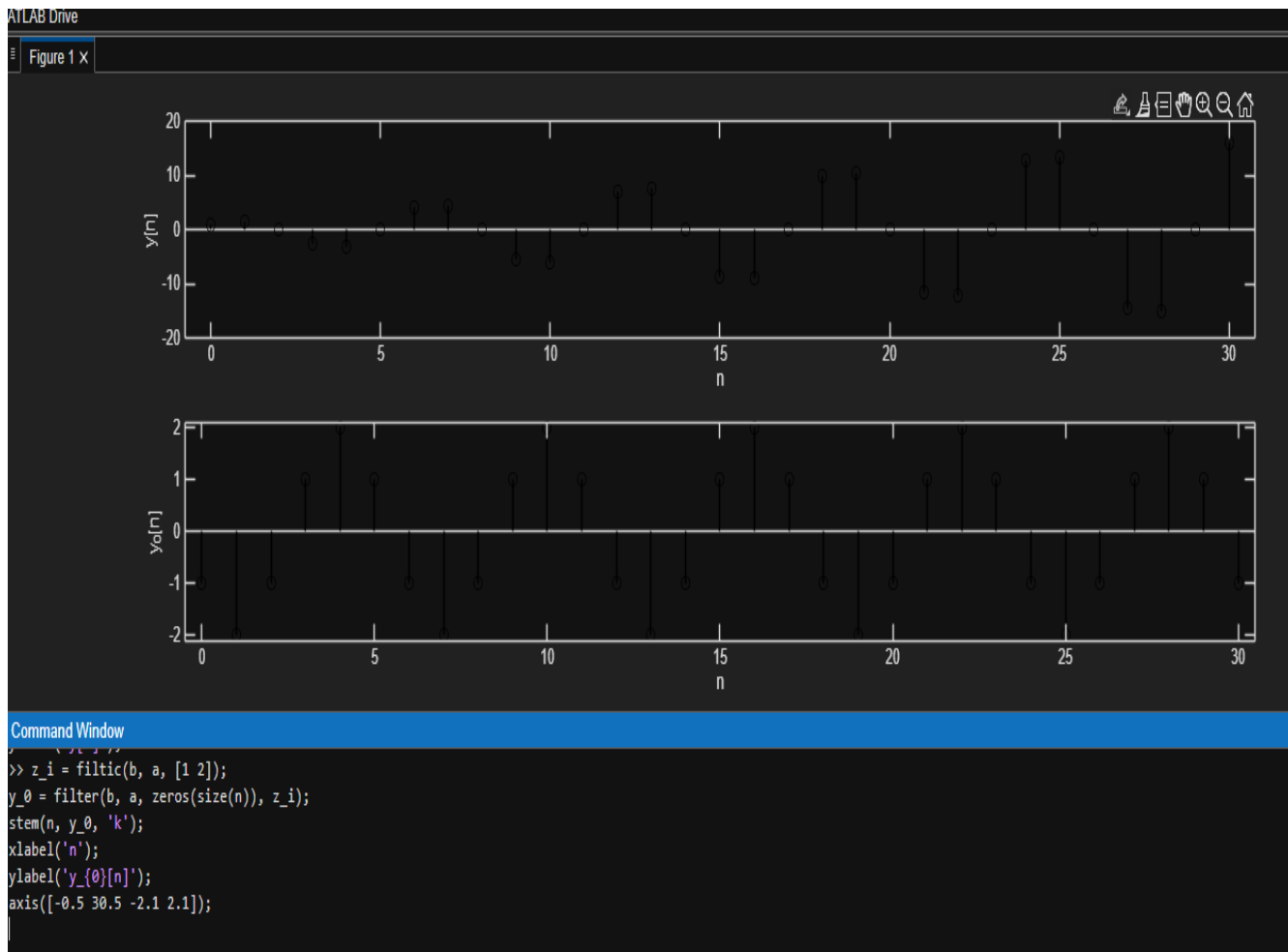
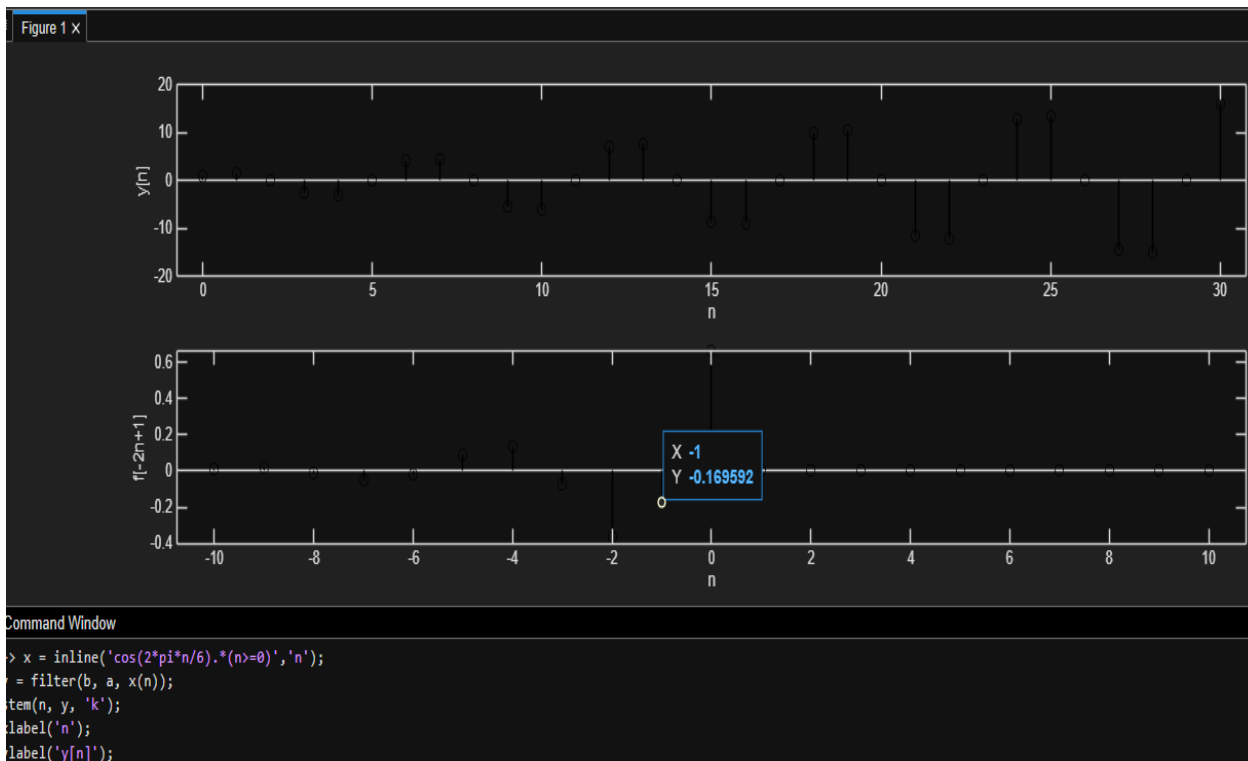
Discrete-time signals were generated and visualized using stem plots, accurately reflecting the expected shapes and time shifts. The `filter` function was used to compute impulse, zero-state, zero-input, and total responses correctly. The total response equaled the sum of the zero-state and zero-input responses, confirming system linearity. All MATLAB outputs in the screenshots aligned with these results.

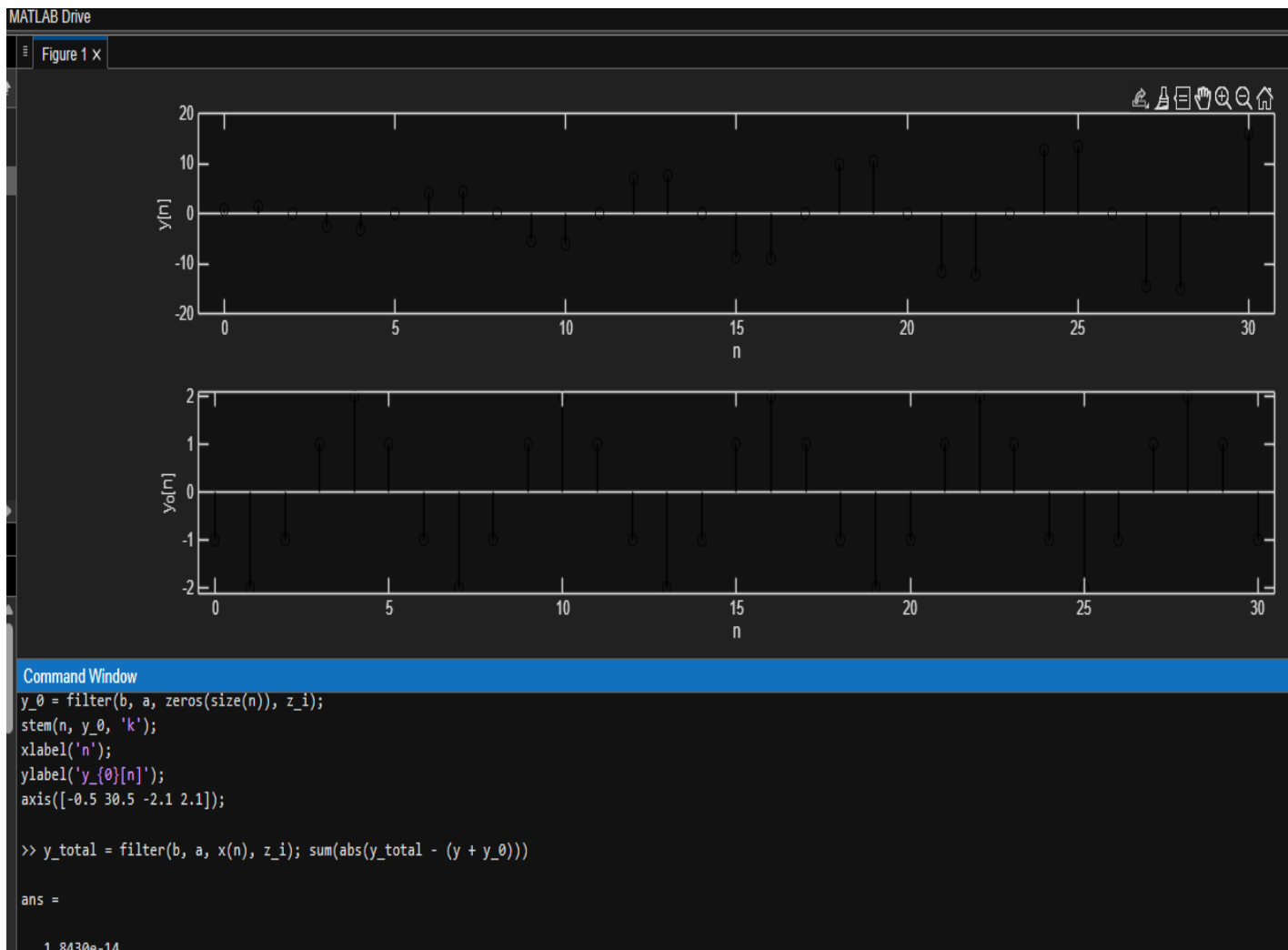
Discrete-Time Functions and Stem Plots



(The lines are in black, I couldn't change it here)







Conclusion

This experiment involved generating and analyzing discrete-time signals and LTI system responses using MATLAB. Stem plots visualized the time-domain sequences, while the `filter` function computed the impulse, zero-state, zero-input, and total responses. The total response equaled the sum of the zero-state and zero-input responses, confirming the system's linearity and time-invariance. The experiment demonstrated MATLAB's effectiveness in practical analysis of discrete-time systems.